

## CHATME API

# Socket.IO Testing with Postman

## Student Testing Guide

Authentication, presence, typing, message replies, and durable receipts

|          |        |          |         |          |
|----------|--------|----------|---------|----------|
| PRESENCE | TYPING | MESSAGES | REPLIES | RECEIPTS |
|----------|--------|----------|---------|----------|

Version 1.1 | 12 August 2026

# Quick start

Use this guide as a practical lab: prepare two users, connect two Socket.IO tabs, and verify every realtime behavior against the durable REST API.

## What you need

- Postman desktop app with Socket.IO client version v4.
- Two authenticated ChatMe users and their current access tokens.
- One direct conversation shared by those users.
- The importable environment file: [ChatMe-Socket.IO.postman\\_environment.json](#).

## Important protocol rule

Production REST requests use https://. Production Socket.IO connections use wss://, the encrypted WebSocket equivalent. The Socket.IO namespace is /chat; the handshake path remains /socket.io.

## Postman limitation

Socket.IO requests belong in a separate multi-protocol collection. Login, conversation creation, message writes, reply writes, and receipt writes remain HTTP/REST operations.

# 1. Import the environment

| Variable                    | Value                                                          |
|-----------------------------|----------------------------------------------------------------|
| apiBaseUrl                  | Local: http://localhost:3000/v1<br>Production: https://HOST/v1 |
| socketUrl                   | Local: ws://localhost:3000/chat<br>Production: wss://HOST/chat |
| accessTokenA / accessTokenB | Current access tokens for two test users                       |
| conversationId              | A direct conversation shared by users A and B                  |
| throughMessageId            | An incoming server message ID for receipt testing              |

Replace HOST with the exact deployed hostname. Keep access tokens as Postman local/secret values. Never commit real tokens or share them in a public workspace.

## Prerequisites

- Sign in two users and copy each accessToken.
- Create or open a direct conversation between them.
- Copy the conversation id into conversationId.

## 2. Create the Socket.IO requests

In Postman select New > Socket.IO, enter `{{socketUrl}}`, and add Authorization: Bearer `{{accessTokenA}}` before connecting.

### Settings

- Socket.IO client version: v4.
- Handshake path: /socket.io.
- Namespace: already present in socketUrl as /chat.
- Render free-tier handshake timeout: about 90000 ms.
- Duplicate the tab for user B and use accessTokenB.

### Authentication failure

```
{
  "code": "AUTH_ACCESS_TOKEN_INVALID",
  "message": "A valid access token is required."
}
```

## 3. Add event listeners

Open Events and select Listen for all six server events before triggering REST requests.

| Event             | When received                                          |
|-------------------|--------------------------------------------------------|
| message.created   | A participant persists a message through REST          |
| receipt.delivered | A durable delivery boundary advances                   |
| receipt.read      | A durable read boundary advances                       |
| presence.changed  | A watched participant becomes online or offline        |
| typing.started    | Another watched participant starts or refreshes typing |
| typing.stopped    | Typing stops explicitly or expires                     |

### message.created

A normal message always includes explicit null reply fields:

```
{
  "id": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "clientMessageId": "74bd9a1d-3b48-49da-87cc-6a79e77140a3",
  "senderId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
  "kind": "text",
  "text": "Hello from Postman",
  "replyToMessageId": null,
  "replyTo": null,
  "createdAt": "2026-08-12T16:30:00.000Z"
}
```

## Reply projection

For a reply, `replyToMessageId` is populated and `replyTo` contains only a safe current-text preview - never the target's full internal record:

```
{
  "replyToMessageId": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
  "replyTo": {
    "id": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
    "senderId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
    "kind": "text",
    "preview": "Hello from Postman"
  }
}
```

## Receipt and chat-state events

### receipt.delivered / receipt.read

```
{
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
  "throughMessageId": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
  "at": "2026-08-12T16:31:00.000Z",
  "version": 2,
  "delivered": {
    "messageId": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
    "at": "2026-08-12T16:30:30.000Z"
  },
  "read": {
    "messageId": "a47d0ec7-fba4-4bdf-9aa4-7639f6ec0f70",
    "at": "2026-08-12T16:31:00.000Z"
  }
}
```

### presence.changed

```
{
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
  "status": "online",
  "occurredAt": "2026-08-12T16:32:00.000Z"
}
```

### typing.started / typing.stopped

```
{
  "conversationId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee",
  "expiresAt": "2026-08-12T16:32:05.000Z"
}
```

## 4. Emit client commands

For every command, enter the exact event name, add one JSON argument, turn on Ack, and inspect the acknowledgement.

### presence.subscribe

```
{
  "conversationId": "{{conversationId}}"
}

{
  "ok": true,
  "data": {
    "conversationId": "550e8400-e29b-41d4-a716-446655440000",
    "participants": [
      {"userId": "0ab53389-d1c7-4903-9c59-ceb5b8a5dbaf", "status": "online"},
      {"userId": "956d3268-0f92-4bc1-a2bb-9c4768ee11ee", "status": "online"}
    ],
    "typing": []
  }
}
```

### presence.unsubscribe

```
{
  "conversationId": "{{conversationId}}"
}
```

### typing.start

```
{
  "conversationId": "{{conversationId}}"
}
```

Typing expires after five seconds. Refresh typing.start every three seconds while input continues.

### typing.stop

```
{
  "conversationId": "{{conversationId}}"
}
```

### Failed acknowledgement

```
{
  "ok": false,
  "error": {
    "code": "REALTIME_CONVERSATION_NOT_FOUND",
    "message": "Conversation was not found."
  }
}
```

Stable error codes: AUTH\_ACCESS\_TOKEN\_INVALID, REALTIME\_PAYLOAD\_INVALID, REALTIME\_CONVERSATION\_NOT\_FOUND, REALTIME\_RATE\_LIMITED, and REALTIME\_INTERNAL\_ERROR.

## 5. Complete two-user test

1. Connect users A and B in separate Socket.IO tabs.
2. Send presence.subscribe from both tabs with the same conversationId.
3. Send typing.start and typing.stop from A. Only B receives the typing events.
4. As A, send a durable message through REST:

```
POST {{apiBaseUrl}}/conversations/{{conversationId}}/messages
Authorization: Bearer {{accessTokenA}}

{
  "clientMessageId": "7d444840-9dc0-41d1-b245-5ffdce74fad2",
  "text": "Hello from Postman"
}
```

Both socket tabs receive one message.created event. Copy A's returned server message id.

5. As B, send a reply using a fresh client UUID:

```
POST {{apiBaseUrl}}/conversations/{{conversationId}}/messages
Authorization: Bearer {{accessTokenB}}

{
  "clientMessageId": "7d444840-9dc0-41d1-b245-5ffdce74fad3",
  "text": "Replying from Postman",
  "replyToMessageId": "<A's returned server message id>"
}
```

Both tabs receive one event with the same replyToMessageId and shallow replyTo projection. Reusing the client UUID with a different or omitted reply target returns 409 MESSAGE\_IDEMPOTENCY\_CONFLICT.

6. As B, advance delivery for A's incoming message:

```
PUT {{apiBaseUrl}}/conversations/{{conversationId}}/receipts/delivered
Authorization: Bearer {{accessTokenB}}

{
  "throughMessageId": "{{throughMessageId}}"
}
```

7. Repeat with /receipts/read. Both tabs receive receipt.read.
8. Disconnect all B sockets. After the ten-second grace period A receives presence.changed with status offline.

## Limits and troubleshooting

- A socket may subscribe to at most 20 conversations.
- A user may have at most five active socket connections by default.
- The server accepts 30 realtime commands per 10-second window per socket.
- Command payloads are strict and may contain only conversationId.
- Production connections use wss://, never https:// or ws://.
- Socket events are live notifications, not durable replay. Reconcile messages and receipts through REST after reconnect.
- A reply target must belong to the same conversation; missing and cross-conversation targets return MESSAGE\_NOT\_FOUND.
- On Render free tier, the first connection may be delayed while the service wakes.

### Reconnect order

Refresh or sign in, reconnect with the new access token, load missed message history, load receipt frontiers, then call presence.subscribe for the open chat.

### Official Postman references

[Create a Socket.IO request](#)

[Listen to Socket.IO events](#)

[Work with WebSocket messages](#)

### Classroom scope

Presence and typing state are process-local in this milestone. Horizontal scaling requires a shared Socket.IO adapter plus shared presence/typing coordination. Durable messages, replies, and receipts remain PostgreSQL-backed.

## Ready to test

Import the environment, connect two users, add all listeners, then work through the two-user flow from top to bottom.